Lecture 19 (B18): PINNs for Economic PDEs — Boundary Conditions, HJB, Black–Scholes

Soft vs hard BCs, HJB cake-eating, DGM architecture, Black–Scholes pricing

Simon Scheidegger

HEC, University of Lausanne

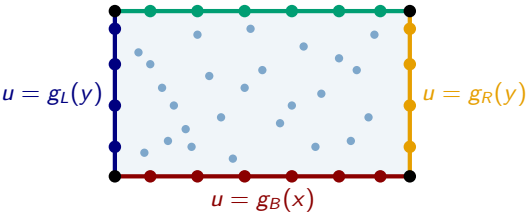
What Are Boundary Conditions?

1D: $y''(x) = -1$ **on** $(0, 1)$

Interior: enforce $y'' + 1 = 0$

Boundary: enforce $y =$ prescribed value

2D: $\Delta u = f$ **on** $(0, 1)^2$

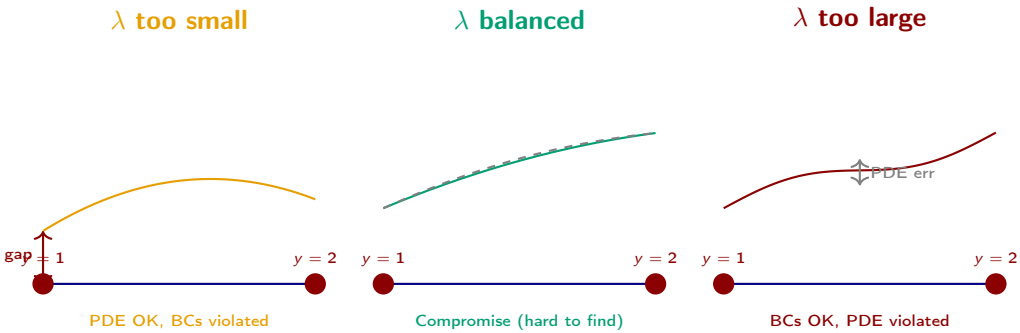


Two enforcement strategies

Soft: add BC violation as a penalty term in the loss.

Hard: build BCs into the network output by construction.

The Problem with Soft Boundary Conditions



Soft BCs require careful balancing of the penalty weight λ.
Too small: BCs violated. Too large: PDE ignored. Just right: hard to find *a priori*.

Hard BC Enforcement: Trial Solutions

Idea: construct a trial solution that satisfies BCs *by construction*:

$$\boxed{\hat{y}(x) = A(x) + B(x) \cdot \mathcal{N}_\theta(x)}$$

where:

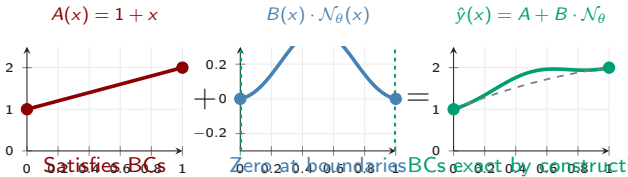
- $A(x)$: anchor function satisfying the BCs exactly, e.g. $A(x) = 1 + x$
- $B(x)$: mask function vanishing at boundaries, e.g. $B(x) = x(1 - x)$
- $\mathcal{N}_\theta(x)$: free neural network (no BC constraints)

Verification:

$$\hat{y}(0) = A(0) + B(0) \cdot \mathcal{N}_\theta(0) = 1 + 0 = 1 \quad \checkmark$$
$$\hat{y}(1) = A(1) + B(1) \cdot \mathcal{N}_\theta(1) = 2 + 0 = 2 \quad \checkmark$$

$\Rightarrow$ BCs hold identically—only the PDE residual loss remains.

Trial Solution: Visual Decomposition



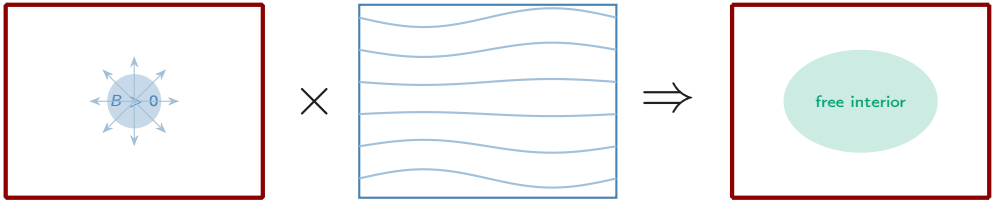
Hard BC: Implementation

```
def A(x): # Anchor: A(0)=1, A(1)=2
    return 1.0 + x
def B(x): # Mask: B(0)=B(1)=0
    return x * (1.0 - x)

def y_trial(model, x): # Hard-BC trial solution
    return A(x) + B(x) * model(x)

def residual(model, x):
    x.requires_grad_(True)
    y = y_trial(model, x)
    dydx = torch.autograd.grad(y, x, ...)[0]
    d2ydx2 = torch.autograd.grad(dydx, x, ...)[0]
    return d2ydx2 + 1.0 # should be 0
```

Advantage: the loss has only one term (PDE residual)—no λ to tune!

Hard BCs in 2D: Building the Trial Solution

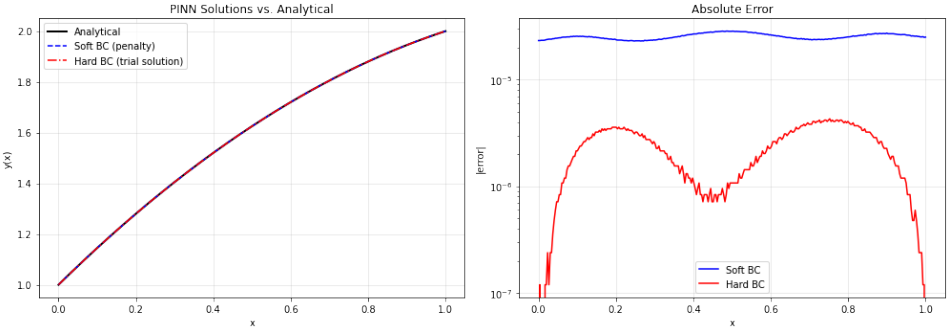
Mask $B(x,y) = x(1-x)\,y(1-y)$ **Network** $\mathcal{N}_\theta(x,y)$ $B \cdot \mathcal{N}_\theta$**: zero on** $\partial\Omega$



$B = 0$ on all edges Free to take any value Exactly zero on boundary

Then $\hat{u}(x,y) = \underbrace{A(x,y)}_{\text{matches BCs}} + \underbrace{B(x,y)\cdot\mathcal{N}_\theta(x,y)}_{\text{zero on } \partial\Omega}$ satisfies Dirichlet BCs exactly.

Soft vs. Hard BCs: Comparison



Hard BCs eliminate boundary error by construction, so the optimizer only fits the PDE interior. The notebook compares this against the tuned soft-penalty baseline.

Notebook: `02_ODE_PINN_SoftVsHardBCs`

PAUSE Finger Exercise: Hard Boundary Conditions

Exercise (5 min)

Consider the same ODE $u''(x) = -1$ on $[0, 1]$ with $u(0) = u(1) = 0$.

1. Construct a hard-BC trial solution of the form $\hat{u}(x) = B(x) \cdot \mathcal{N}(x; \theta)$, where $B(x)$ enforces both BCs exactly. (Hint: B must vanish at $x = 0$ and $x = 1$.)
2. Verify that $\hat{u}(0) = \hat{u}(1) = 0$ for *any* network output $\mathcal{N}$.
3. What is the advantage of hard BCs over soft BCs?

Solution: Hard Boundary Conditions

Answer

1. $\hat{u}(x) = x(1-x) \cdot \mathcal{N}(x;\theta)$. Since $B(x) = x(1-x)$ vanishes at both endpoints, the BCs are satisfied exactly.
2. $\hat{u}(0) = 0 \cdot \mathcal{N}(0;\theta) = 0. \quad \hat{u}(1) = 0 \cdot \mathcal{N}(1;\theta) = 0.$ ✓
   This holds regardless of what $\mathcal{N}$ outputs.
3. Hard BCs eliminate the boundary loss term entirely—the optimizer only needs to minimize the PDE residual. This reduces the number of hyperparameters (λ is gone) and often speeds up convergence.

Extension to 2D: Poisson Equation

Problem: $\Delta u = f(x, y)$ on $\Omega = (0, 1)^2$ with Dirichlet BCs.

Manufactured solution: $u^\star(x, y) = x^2 + y + \sin(\pi x)\sin(\pi y)$.

Hard BCs via transfinite (Coons-patch) interpolation — the standard 4-side construction that matches each boundary function exactly along its edge:

$$\hat{u}(x, y) = A(x, y) + \underbrace{x(1-x)\,y(1-y)}_{B(x,y)} \cdot \mathcal{N}_\theta(x, y)$$

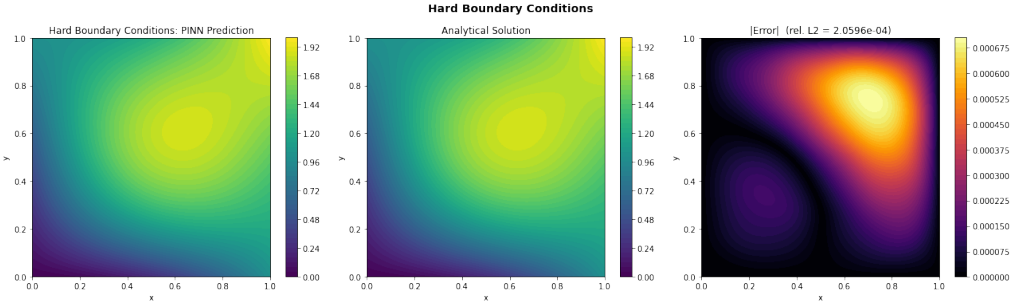
where $A(x, y)$ interpolates boundary data from all four sides:

$$A = (1-x)\,g_L(y) + x\,g_R(y) + (1-y)\,g_B(x) + y\,g_T(x) - \text{corners}$$

Laplacian via autograd:

$$\Delta \hat{u} = \frac{\partial^2 \hat{u}}{\partial x^2} + \frac{\partial^2 \hat{u}}{\partial y^2} \quad \text{(two chains of } \texttt{autograd.grad}\text{)}$$

2D Poisson: Results



Hard-BC PINN, analytical solution, and absolute error.
Relative L^2 error $\approx 2.06 \times 10^{-4}$. Boundary conditions satisfied exactly.

Notebook: `03_PDE_PINN_Poisson2D`

Soft vs. Hard BCs: When to Use Which?

	Soft BCs	**Hard BCs**
Best for	Complex geometries, time-dependent BCs, Robin/Neumann	Simple domains, Dirichlet BCs, high accuracy
Pros	Flexible, easy to implement	Exact BCs, fewer hyperparams, better accuracy
Cons	λ tuning, approximate BCs	Needs analytical A, B; harder for irregular domains

Practical recommendation

Start with hard BCs when the domain and boundary data allow it. Fall back to soft BCs for complex geometries or mixed boundary types.

Part IV

Economic Applications: The HJB Equation

The Hamilton–Jacobi–Bellman Equation

Generic continuous-time optimization:

$$\rho\, V(\boldsymbol{x}) = \max_{c} \left\{ u(c) + V_{\boldsymbol{x}} \cdot f(\boldsymbol{x}, c) \right\}$$

- $V(\boldsymbol{x})$: value function (unknown)
- ρ: discount rate
- $u(c)$: flow utility
- $f(\boldsymbol{x}, c)$: law of motion for state $\boldsymbol{x}$
- $V_{\boldsymbol{x}} = \nabla V$: gradient of the value function

This is a PDE! First-order (or second-order with diffusion).

$\Rightarrow$ Perfect fit for PINNs: the network approximates V, autograd gives $V_{\boldsymbol{x}}$, and the HJB residual is the loss.

Cake-Eating Problem: Setup

A household chooses consumption $c(t)$ to maximize lifetime utility:

$$\max_{\{c(t)\}} \int_0^\infty e^{-\rho t} \frac{c(t)^{1-\gamma}}{1-\gamma} \, \mathrm{d}t \qquad \text{s.t.} \quad \dot{a} = r\,a - c$$

HJB equation: $\rho\, V(a) = \max_c \left\{ \frac{c^{1-\gamma}}{1-\gamma} + V'(a)\,(r\,a - c) \right\}$

FOC: $c^{-\gamma} = V'(a) \;\Rightarrow\; c^\star = \big(V'(a)\big)^{-1/\gamma}$.

Analytical solution (with $\kappa = \frac{\rho - (1-\gamma) r}{\gamma} > 0$):

$$V^\star(a) = \frac{\kappa^{-\gamma}}{1-\gamma}\, a^{1-\gamma}, \qquad c^\star(a) = \kappa\, a$$

Parameters: $\gamma = 2$, $\rho = 0.05$, $r = 0.03 \;\Rightarrow\; \kappa = 0.04$.

PINN for the HJB Equation: Hard-BC Trial Solution

Anchor + mask + scaled MLP. Let $x(a) = (a - a_{\min})/(a_{\max} - a_{\min}) \in [0, 1]$ and S a value-scale constant:

$$\hat{V}(a) \;=\; V^\star(a_{\min}) + x(a)\big[V^\star(a_{\max}) - V^\star(a_{\min})\big] \;+\; x(a)\big(1 - x(a)\big)\, S\, f_\theta\big(x(a)\big).$$

Endpoints are exact *by construction*, so the boundary loss is identically zero and the optimizer focuses on the HJB residual only.

Consumption via FOC, $\hat{c}(a) = (\hat{V}'(a))^{-1/\gamma}$, with softplus safeguard on $\hat{V}'$.

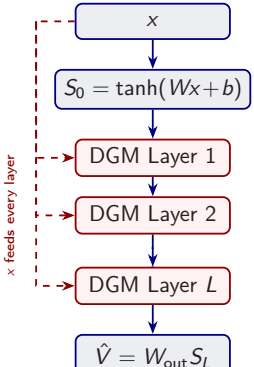
Loss: only the interior HJB residual remains,

$$\ell \;=\; \frac{1}{N}\sum_i \mathcal{R}(a_i)^2, \qquad \mathcal{R}(a) = \rho\hat{V} - \left[\tfrac{\hat{c}^{1-\gamma}}{1-\gamma} + \hat{V}'(ra - \hat{c})\right].$$

Pedagogical point: a small MLP plus hard BCs beats a large gated net with a soft-BC penalty, because the boundary term no longer dominates the objective.

Optional Architecture for High-D PDEs: DGM

Deep Galerkin Method (Sirignano & Spiliopoulos, 2018) — a heavier architecture that becomes worth its weight only in higher dimensions or with sharp PDE features. For the 1D cake problem, the trial-solution MLP on the previous slide is preferred.



DGM Layer gates (LSTM-like, Hochreiter & Schmidhuber 1997); x-skip connections recall Highway Networks (Srivastava et al. 2015). $S_\ell \in \mathbb{R}^U$ is the depth-ℓ hidden state ($S_0 = \tanh(Wx + b)$), $\odot$ is Hadamard, Z, G, R are sigmoid gates in $[0, 1]$:

$$
\begin{aligned}
Z_\ell &= \sigma(U^z x + W^z S_\ell + b^z) \\
G_\ell &= \sigma(U^g x + W^g S_\ell + b^g) \\
R_\ell &= \sigma(U^r x + W^r S_\ell + b^r) \\
H_\ell &= \tanh(U^h x + W^h (S_\ell \odot R_\ell) + b^h) \\
S_{\ell+1} &= (1 - G_\ell) \odot H_\ell + Z_\ell \odot S_\ell
\end{aligned}
$$

Key idea: feeding x into every layer helps the network resolve sharp PDE gradients.

DGM Implementation (PyTorch)

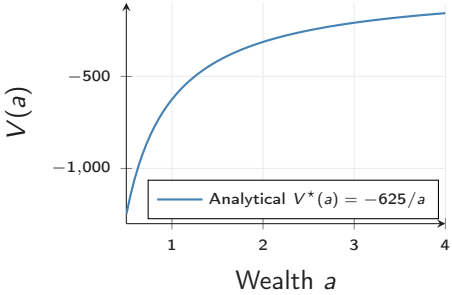
```
class DGMNet(nn.Module):
    def __init__(self, dim, layers=4, units=64):
        super().__init__()
        self.input = nn.Linear(dim, units)
        self.z = nn.ModuleList(nn.Linear(dim, units) for _ in range(layers))
        self.wz = nn.ModuleList(nn.Linear(units, units) for _ in range(layers))
        # ... similar for g, r, h gates
        self.out = nn.Linear(units, 1)

    def forward(self, x):
        s = torch.tanh(self.input(x))
        for l in range(len(self.z)):
            z = torch.sigmoid(self.z[l](x) + self.wz[l](s))
            g = torch.sigmoid(self.g[l](x) + self.wg[l](s))
            r = torch.sigmoid(self.r[l](x) + self.wr[l](s))
            h = torch.tanh(self.h[l](x) + self.wh[l](s * r))
            s = (1 - g) * h + z * s
        return self.out(s)
```

HJB Residual Implementation

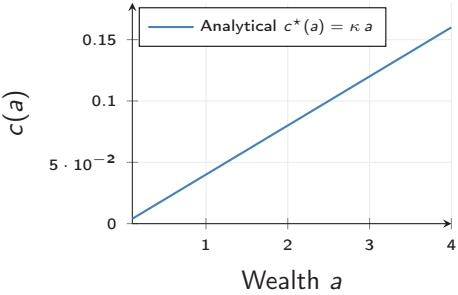
```python
def pde_residual(model, a):
    a.requires_grad_(True)
    V = model(a) # value function
    V_a = torch.autograd.grad( # V'(a) via autograd
        V.sum(), a, create_graph=True)[0].squeeze(-1)

    safe_Va = F.softplus(V_a) + 1e-6 # positivity guard
    c = safe_Va.pow(-1.0 / gamma) # FOC: optimal c
    u_c = c.pow(1-gamma) / (1-gamma) # flow utility

    R = rho*V - (u_c + safe_Va*(r*a.squeeze(-1) - c))
    return R # HJB residual
```

The `softplus` safeguard prevents $V'(a)^{-1/\gamma}$ from blowing up if $V'(a) \leq 0$ during early training.

Cake-Eating: Expected Behavior (Schematic)



Value function $V(a)$ (closed form)



Consumption policy $c(a)$ (closed form)

Validation target: closed form. Classroom run: final HJB loss $\approx 3\times10^{-4}$; consumption relative L^2 error $\approx 6\times10^{-3}$ after Adam + FP64 L-BFGS.

Notebook: `04_Cake_Eating_HJB_PINN`

Extensions: From Cake-Eating to Frontier Models

The PINN/DGM framework scales naturally:

1. **Income risk:** add diffusion $\mathrm{d}a = (ra - c + y)\,\mathrm{d}t + \sigma\,\mathrm{d}W$ $\Rightarrow$ second-order HJB
2. **Aiyagari-type models:** HJB + Kolmogorov Forward Equation $\Rightarrow$ coupled PDE system
3. **General equilibrium:** prices clear markets $\Rightarrow$ hybrid DEQN + PINN
4. **High dimensions:** multiple assets, heterogeneous agents $\Rightarrow$ no grid needed!

See: Achdou et al. (2022), Fernandez-Villaverde et al. (2024), Gopalakrishna (2024).

Part V

Financial Applications: Black–Scholes

The Black–Scholes PDE

European call option (S: asset, K: strike, T: maturity):

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0$$

Terminal: $V(S,T) = \max(S-K,0)$

Boundary conditions:

- $V(0,t) = 0$
- $V(S_{\max},t) = S_{\max} - Ke^{-r(T-t)}$

PINN approach:

- Network $\mathcal{N}_\theta(S,t) \approx V(S,t)$
- Autograd: V_t, V_S, V_{SS}
- 4 loss terms:

ℓ_{PDE}	interior residual
ℓ_{BC_0}	$V(0,t) = 0$
ℓ_{term}	$V(S,T) = \text{payoff}$
$\ell_{\text{BC}_{S_{\max}}}$	far-field BC

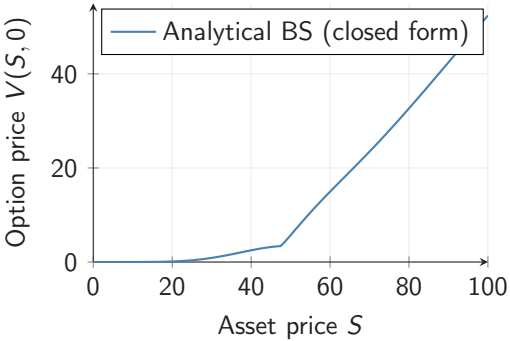
Black–Scholes PINN: Key Code

```
def pde_residual(model, S, t):
    S.requires_grad_(True)
    t.requires_grad_(True)
    X = torch.cat([S, t], dim=1)
    V = model(X)

    V_t = torch.autograd.grad(V, t, ...)[0]
    V_S = torch.autograd.grad(V, S, ...)[0]
    V_SS = torch.autograd.grad(V_S, S, ...)[0]

    # Black-Scholes PDE residual
    res = V_t + 0.5*sigma**2*S**2*V_SS + r*S*V_S - r*V
    return res
```

Loss:

$$\ell = \mathrm{MSE}(\mathrm{res}) + \mathrm{MSE}(V|_{S=0}) + \mathrm{MSE}(V|_{t=T} - \mathrm{payoff}) + \mathrm{MSE}(V|_{S_{\max}} - \mathrm{target})$$

Black–Scholes: Expected Behavior (Schematic)



Parameters: $K = 50$, $T = 1$, $r = 0.05$, $\sigma = 0.2$, $S_{\max} = 100$.

Network: 3 hidden layers, 50 units, tanh activations, 10,000 epochs (Adam).

Schematic of the call-option price curve. Current notebook diagnostics report max absolute error ≈ 0.13 and mean absolute error ≈ 0.04 against the closed form.

Notebook: `05_Black_Scholes_PINN`

Summary and Key Takeaways

Method

PINNs embed differential equations directly into the neural network loss function via automatic differentiation.

Boundary Conditions

Hard BCs (trial solutions) eliminate tuning and ensure exact satisfaction. Use soft BCs for complex domains.

Applications

HJB equations, Black–Scholes, and any continuous-time model. Scales to high dimensions without grids.

From DEQNs to PINNs:

- Same philosophy: economic/financial equations *are* the loss
- New ingredient: `torch.autograd.grad` for derivatives of $\mathcal{N}_\theta$
- Broader scope: any PDE, not just algebraic equilibrium conditions

Questions?

Simon Scheidegger

HEC, University of Lausanne

https://sischei.github.io

Materials: https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models

Companion Notebooks:

01 ODE PINN (Zero BCs)
02 Soft vs. Hard BCs
03 2D Poisson
04 Cake-Eating HJB
05 Black–Scholes

Next up (May 5th):

Continuous-Time Heterogeneous Agent Models: HJB + KFE

09:00–12:00